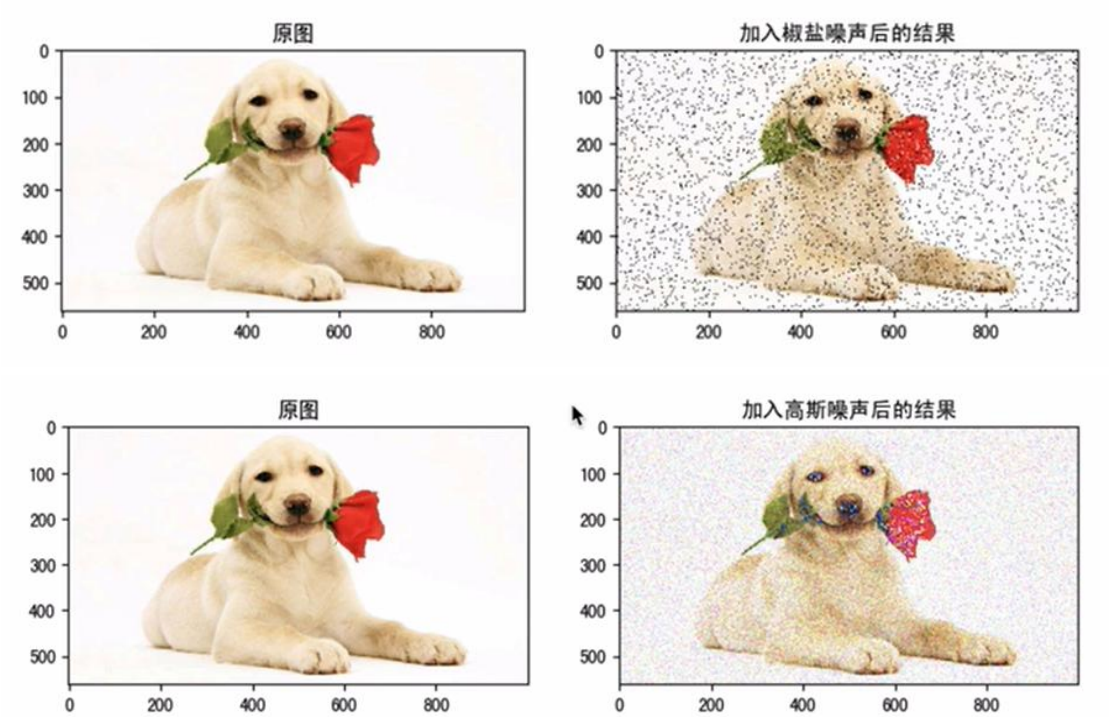
一、图像中的噪声

在智慧城市中图像噪声是指存在于图像数据中的不必要的或多余的干扰信息，图像噪声的产生来自图像获取的环境条件和传感元器件自身的质量，图像在传输过程中产生图像噪声的主要因素是所用的传输信道收到了噪声的污染。

1.**椒盐噪声**也称为脉冲噪声，是图像中经常见到的一种噪声，所谓椒盐，椒就是黑，盐就是白。它是一种随机出现的白点或者黑点，可能是明亮的区域有黑色像素或是在阴暗的区域有白色像素，或是两者皆有。

2.**高斯噪声**是指它的概率密度函数服从高斯分布（即正态分布）的一类噪声（各种灰度噪声）。高斯噪声也是数字图像的一个常见噪声。主要来自电路各元器件自身噪声和相互影响高斯噪声是所有噪声当中分布最为广泛的，传感器在低照明度或者高温的条件下产生的噪声就属于高斯噪声，电子电路中产生的噪声也属于高斯噪声，还有很多噪声都可以根据高斯分布（正态分布）的形式进行描述。



二、图像滤波的原理

图形滤波的操作可以理解为，使用滤波器(卷积核)在图像上进行滑动，通过与图像中的像素值进行加权组合以生成输出图像，不同的滤波器可以实现不同的效果。图像也可以看成一种数字信号，在信号处理中通常使用滤波器，在深度学习中通常使用卷积和这个概念。

1.卷积核滤波器

卷积核滤波器也叫卷积核、滤波核、滤波器，通常是一个矩阵(3×3 或者 5×5)，用图像数据进行卷积计算。通常，卷积核的宽度和高度一般是奇数，这样才有中心的像素点，所以卷积核一般都是 3×3，5×5 或者 7×7 等。$n×n$的卷积核的半径为$(n-1)/2$，例如 5×5 大小的卷积核的半径就是 2，例如定义一个平均滤波器，如图所示。

$$K = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

定义一个平均滤波器

2. 卷积运算

数字图像是一个二维的数组，对数字图像做卷积操作其实就是利用卷积核在图像上滑动，将图像点上的像素值与对应的卷积核上的数值相乘，然后将所有相乘后的值相加作为卷积核中间像素点的像素值，并最终滑动完成所有图像的过程，卷积运算的公式可以表示如式。

$$O(x,y) = \sum_s \sum_t I(x-s, y-t).K(s,t)$$

其中，输入图像为 $I(x,y)$，输出图像为 $O(x,y)$，卷积核为 $K(s,t)$，输出图像中的每个像素 O(x,y)都是通过将卷积核 K 与输入图像 I 上对应位置的局部区域进行卷积运算而得到的。公式中的两个求和符号分别在卷积核的所有行和列上执行。

例如：使用平均滤波器对图像进行处理，将核放在图像的一个像素 A 上，求与核对应的图像上 25（5×5）个像素的和，再取平均数，用这个平均数替代像素 A 的值。重复以上操作直到将图像的每一个像素值都更新，输入图像如图所示。

INPUT IMAGE

18	54	51	239	244	188
55	121	75	78	95	88
35	24	204	113	109	221
3	154	104	235	25	130
15	253	225	159	78	233
68	85	180	214	245	0

3. 图像滤波分类

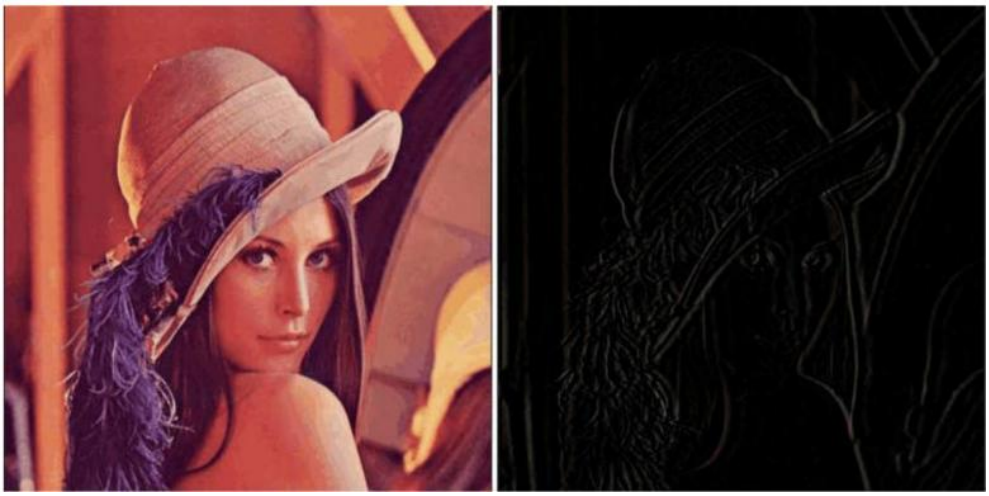
低频（Low Frequency）：指的是图像中变化缓慢的区域，例如大面积的平滑区域或渐变色彩。这些区域在频域中表现为低频分量。

高频（High Frequency）：指的是图像中变化迅速的区域，例如边缘、细节或噪声。这些区域在频域中表现为高频分量。

低频滤波 LPF 使得图像的高频区域变成低频，即色彩变换剧烈的区域变得平滑，处理后出现模糊效果。

低频滤波

高频滤波 HPF，和高频正好相反，过滤了低频，只保留那些变换最快速最剧烈的区域，也就是图像里面的物体边缘，所以常用于边缘识别。



高频滤波

三、图像滤波操作

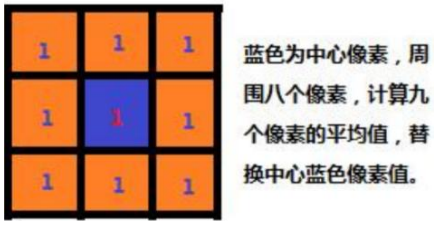
1.均值图像滤波

主要用于图像的平滑操作，用于消除噪音、模糊处理，是低频增强的空间滤波技术，一般采用简单平均法计算，即求邻近像素的平均亮度值。相邻区域的大小与平滑的效果有关，邻域越大，平滑效果越好，但是邻域太大会导致边缘信息损失越大，从而使处理后的图像过于模糊。

从频率域角度出发，均值滤波是一种低通滤波器，该滤波会去掉高频信号，因此可以帮助消除图像尖锐噪声，实现图像平滑，模糊等功能。

（1）计算方式

理想的均值滤波是用每个像素和它周围像素计算出来的平均值替换图像中每个像素。采样 Kernel 数据通常是 3X3 的矩阵。



（2）均值滤波函数 ： cv2.blur()-均值滤波-取平均值，

默认进行归一化处理（即所有元素之和为 1）

邻域平均法模糊，用于减少图像噪声和细节，以获得平滑的视觉效果

dst = cv2.blur(src, ksize[, anchor[, borderType])

src：输入图像，可以是灰度图或彩色图。

ksize：指定模糊核的大小，是一个元组，其两个元素分别是模糊核的宽度和高度。例如，(5, 5) 表示 5x5 的核。

anchor（可选）：表示核的锚点，用于指定核的参考点。默认值是 (-1, -1)，表示核的中心。

borderType(可选):指定边界模式,用于处理图像边界外的像素。默认值是 cv2.BORDER_DEFAULT，表示使用重复边界像素的方法。

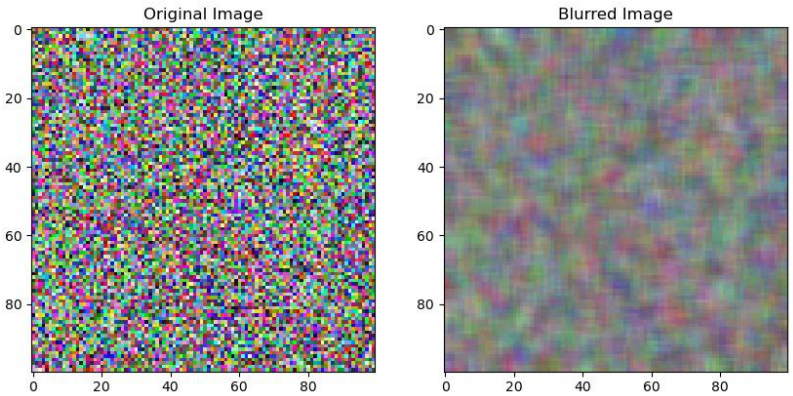
返回值

dst：输出图像，与输入图像 src 具有相同的大小和类型，但经过模糊处理。

示例：dst=cv2.blur(img,(5,5))

举例：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 创建一个示例图像
img = np.random.randint(0, 256, (100, 100, 3), dtype=np.uint8)
# 应用均值模糊
blurred_img = cv2.blur(img, (5, 5))
# 使用 matplotlib 显示原始图像和模糊后的图像
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(blurred_img, cv2.COLOR_BGR2RGB))
plt.title("Blurred Image")
plt.show()
```



（3）cv2.boxFilter()-方框滤波-取平均值，不做归一化处理

通过卷积操作将每个像素替换为其邻域内所有像素的平均值来实现图像平滑的效果。

函数：boxFilter(img, depth, ksize, boolnormalize)

第 1 个参数，img，输入图像，即源图像。

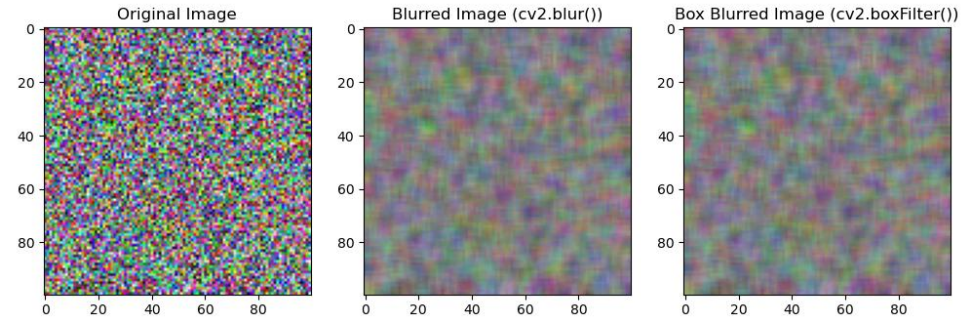
第 2 个参数，int 类型的 depth，输出图像的深度，-1 代表使用原图深度，即 src.depth()。
第 3 个参数，Size 类型的 ksize，内核的大小。一般这样写 Size(w,h)来表示内核的大小(其中，w 为像素宽度， h 为像素高度)。Size（3,3）就表示 3×3 的核大小，Size（5,5）就表示 5×5 的核大小。
第 4 个参数，bool 类型的 normalize，默认值为 true，一个标识符，表示内核是否被其区域归一化（normalized）了。
当 normalize=true 的时候，均值滤波是方框滤波归一化（normalized）后的特殊情况。
其中，归一化就是把要处理的量都缩放到一个范围内,比如(0,1)，以便统一处理和直观量化。
均值滤波本身存在着固有的缺陷，即它不能很好地保护图像细节，在图像去噪的同时也破坏了图像的细节部分，从而使图像变得模糊，不能很理想地去除噪声点。
示例：box=cv2.boxFilter(img,-1,(5,5),normalize=True)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 创建一个示例图像
img = np.random.randint(0, 256, (100, 100, 3), dtype=np.uint8)

# 使用 cv2.blur() 进行模糊处理
blurred_img = cv2.blur(img, (5, 5))

# 使用 cv2.boxFilter() 进行模糊处理
box_blurred_img = cv2.boxFilter(img, -1, (5, 5))

# 使用 matplotlib 显示原始图像和模糊后的图像
plt.figure(figsize=(12, 6))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(blurred_img, cv2.COLOR_BGR2RGB))
plt.title("Blurred Image (cv2.blur())")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(box_blurred_img, cv2.COLOR_BGR2RGB))
plt.title("Box Blurred Image (cv2.boxFilter())")

plt.show()
```

cv2.blur()和 cv2.boxFilter()差别

核的归一化：cv2.blur()默认对核进行归一化处理，而 cv2.boxFilter()不进行归一化。
计算方式：cv2.blur()在计算过程中会将核的元素值除以核的总面积，而 cv2.boxFilter()则直接将邻域像素值相加。

用途：两者都可用于去噪和平滑图像，但 cv2.blur()在使用归一化核时可以保持图像亮度的一致性。cv2.boxFilter()由于没有进行归一化处理，可能会导致图像的亮度轻微增加。



2.中值滤波--cv2.medianBlur，按小到大顺序排列后，取中间值

是 OpenCV 中用于对图像进行中值滤波的函数。中值滤波是一种非线性滤波方法，常用于去除图像中的噪声，尤其是椒盐噪声。它通过将每个像素值替换为邻域内的中值来实现平滑效果。

cv2.medianBlur(src,ksize)

src：输入图像，可以是灰度图像或多通道图像。

ksize：滤波器的大小，必须是正奇数（如 3、5、7 等）。它表示邻域的大小。

返回值：返回中值滤波后的图像。

ksize=3：较弱的平滑效果，适合去除轻微噪声。

ksize=5：中等强度的平滑效果，适合去除中等程度的噪声。

ksize=7 或更大：较强的平滑效果，适合去除较严重的噪声，但可能会模糊图像细节。

椒盐噪声（Salt-and-PepperNoise）是一种常见的图像噪声类型，通常表现为图像中随机出现的白色（盐）和黑色（椒）像素点。这种噪声得名于其外观类似于在图像上撒了盐和胡椒。

邻域：在图像处理中，邻域是指以当前像素为中心的周围像素区域。例如，对于一个 3x3 的邻域，当前像素为中心像素，周围有 8 个邻居像素。

ksize：在 cv2.medianBlur 中，ksize 指定了邻域的大小，必须是正奇数（如 3、5、7 等）。例如，ksize=3 表示 3x3 的邻域，ksize=5 表示 5x5 的邻域。

55	58	22	55	22	60	168	162	232
123	17	66	33	77	68	14	74	67
47	22	97	95	94	25	14	5	76
68	66	93	78	90	171	82	78	65
69	99	66	91	101	200	192	59	74
98	88	88	45	36	119	47	28	5
88	158	3	88	69	211	234	192	120
77	148	25	45	77	173	226	146	213
42	125	135	58	44	51	79	66	3

CSDN @Justth.

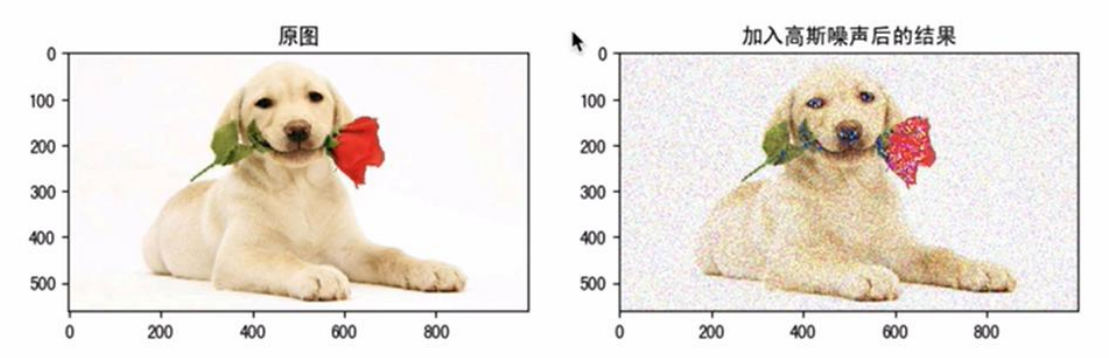
将其邻域设置为 3×3 大小，对其 3×3 邻域内像素点的像素值进行排序（升序降序均可），按升序排序后得到序列值为：[66,78,90,91,93,94,95,97,101]。在该序列中，处于中心位置（也叫中心点或中值点）的值是“93”，因此用该值替换原来的像素值 78，作为当前点的新像素值。

中值滤波效果如下：

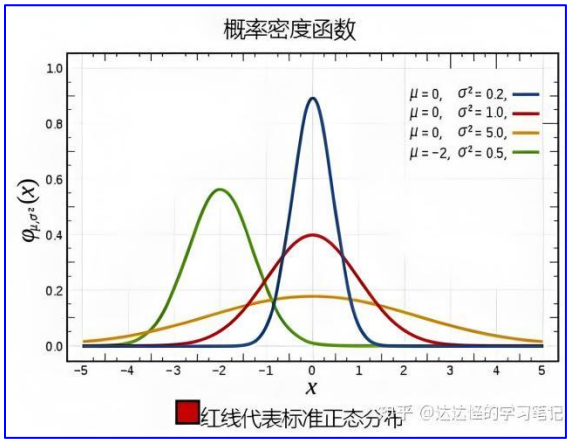
55	58	22	55	22	60	168	162	232
123	17	66	33	77	68	14	74	67
47	22	97	95	94	25	14	5	76
68	66	93	**93**	90	171	82	78	65
69	99	66	91	101	200	192	59	74
98	88	88	45	36	119	47	28	5
88	158	3	88	69	211	234	192	120
77	148	25	45	77	173	226	146	213
42	125	135	58	44	51	79	66	3

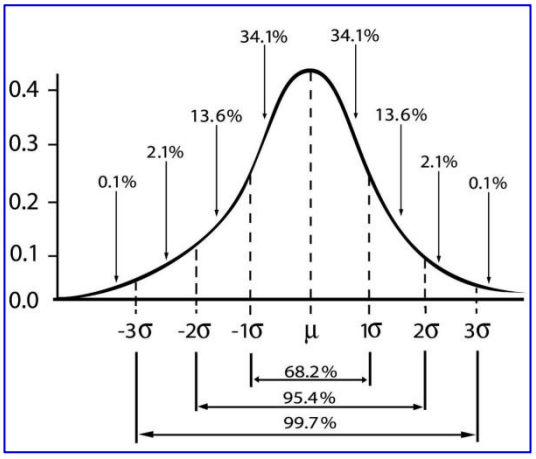
CSDN @justth.

3.高斯滤波-cv2.GaussianBlur 正态分布公式



用于对图像应用高斯模糊（高斯滤波滤波或高斯平滑）处理。高斯模糊是一种线性滤波器使用正态分布（高斯分布）作为滤波器，对图像进行平滑处理，从而减少图像噪声，特别是高斯噪声，同时保持边缘细节。μ，平均值，决定分布位置，σ，标准差，决定分布的宽度。

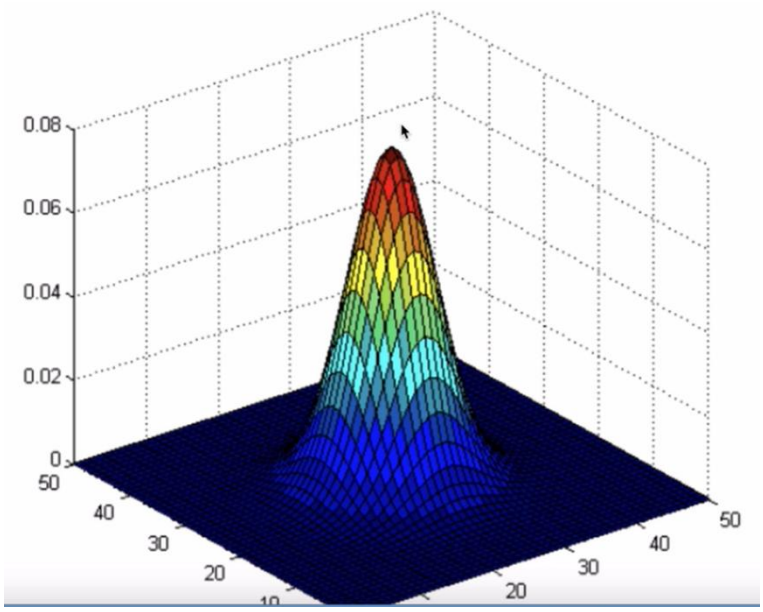




一维高斯函数图像

二维高斯函数

$$G(x,y)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$$



二维高斯函数图像

标准差σ代表着数据的离散程度。

当σ较小，那么生成的模板中心系数较大，而周围的系数较小，这样对图像的平滑效果就不是很明显；

当σ较大，则生成的模板的各个系数相差就不是很大，比较类似均值模板，对图像的平滑效果比较明显。

横轴表示可能的取值 x，竖轴表示概率分布密度 F(x)，曲线与 x 轴围成的图形面积为 1。

σ（标准差）决定了这幅图形的宽度，可以得出结论：

σ越大，则图形越宽，尖峰越小，图形较为平缓；

σ越小，则图形越窄，越集中，中间部分也就越尖，图形变换比较剧烈。

当标准差σ越大，则表示该密度分布一定比较分散，由于面积为 1，于是尖峰部分减小，宽度越宽（分布越分散）；

当标准差σ越小时，说明密度分布较为集中，于是尖峰越尖，宽度越窄。

于是得到如下结论：

σ越大，分布越分散，各部分比重差别不大，于是生成的模板各元素值差别不大，类似于平均模板；

σ越小，分布越集中，中间部分所占比重远远高于其他部分，反映到高斯模板上就是中心元素值远远大于其他元素值，于是相当于中间值的点运算。

正态分布是一种钟形曲线，越接近中心，取值越大，越远离中心，取值越小。计算平滑结果时，只需要将“中心点”作为原点，其他点按照其在正态曲线上的位置，分配权重，就可以得到一个加权平均值。

高斯平滑在从图像中去除高斯噪声方面非常有效。

3 高斯核的计算

高斯核是整个求解的关键。显然，它是通过二维高斯函数计算得来的。这里给出离散高斯核矩阵的计算公式。

$$G(x,y)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$$

其中(x, y)为点坐标，在图像处理中可认为是整数；σ是标准差。要想得到一个高斯滤波器的模板，可以对高斯函数进行离散化，得到的高斯函数值作为模板的系数。

例如：产生一个 3×3 的高斯滤波器模板，需要以模板的中心位置为坐标原点进行取样。模板在各个位置的坐标（x 轴水平向右，y 轴竖直向下）

（1）确定权重矩阵：假定中心点的坐标是(0, 0），那么距离它最近的 8 个点的坐标如图：

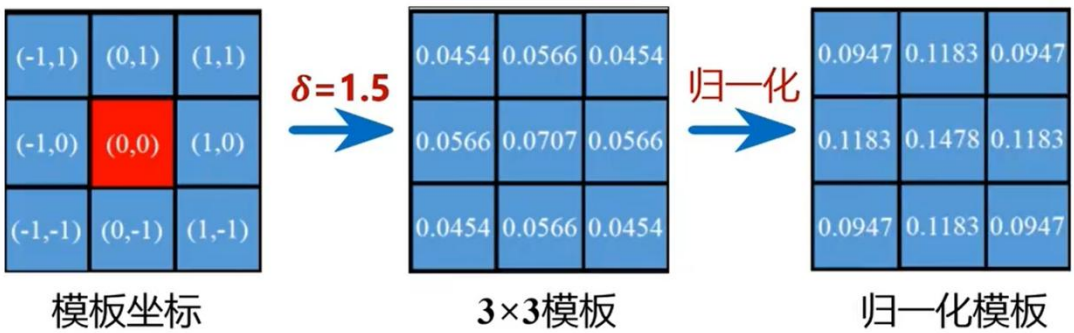
(-1,1)	(0,1)	(1,1)
(-1,0)	(0,0)	(1,0)
(-1,-1)	(0,-1)	(1,-1)

为了计算权重矩阵，需要设定σ的值。假定σ=1.5，则模糊半径为 1 的权重矩阵如下:

0.0453542	0.0566406	0.0453542
0.0566406	0.0707355	0.0566406
0.0453542	0.0566406	0.0453542

这 9 个点的权重总和等于 0.4787147,如果只计算这 9 个点的加权平均，还必须让它们的权重之和等于 1,因此上面 9 个值还要分别除以 0.4787147，得到最终的权重矩阵。

0.0947416	0.118318	0.0947416
0.118318	0.147761	0.118318
0.0947416	0.118318	0.0947416



（2）计算高斯模糊

有了权重矩阵，就可以计算高斯模糊的值了。假设现有 9 个像素点，灰度值(0-255) 如左图，

14	15	16
24	25	26
34	35	36

每个点乘以对应的权重值，得到右图，就是中心店的高斯模糊数据的值，如果原图是彩色图片，可以对 RGB 三个通道分别做高斯平滑。

14x0.0947416	15x0.118318	16x0.0947416
24x0.118318	25x0.147761	26x0.118318
34x0.0947416	35x0.118318	36x0.0947416

1.32638	1.77477	1.51587
2.83963	3.69403	3.07627
3.22121	4.14113	3.4107

dst = cv2.GaussianBlur(src, ksize, sigmaX, sigmaY=None, borderType 默认值)

src：输入图像，可以是灰度图或彩色图。

ksize：指定滤波器的大小，必须是正奇数。例如，(5, 5) 表示 5x5 的滤波器。

sigmaX：X 方向上的标准差，如果为 0，则从 ksize 计算。

sigmaY：Y 方向上的标准差，如果为 None 或 0，则使用与 sigmaX 相同的值。

borderType：指定边界模式，默认为 cv2.BORDER_DEFAULT，即 cv2.BORDER_REFLECT

dst：输出图像，与输入图像 src 具有相同的大小和类型，但经过高斯模糊处理。

```
dst=cv2.GaussianBlur(img,(5,5),1)
```

4. 三种滤波方式小结

1.图像噪声

椒盐噪声:图像中随机出现的白点或者黑点

高斯噪声:噪声的概率密度分布是正态分布

2.图像平滑

均值滤波:算法简单，计算速度快，在去噪的同时去除了很多细节部分，将图像变得模糊.cv.blur()

中值滤波:去除椒盐噪声。cv.medianBlur()

高斯滤波:去除高斯噪声。cv.GaussianBlur()

五、对噪声图片进行测试

1.生成高斯噪声图片

```
img=cv2.imread('../image/dog2.jpg')
img2=cv2.resize(img,(400,600))
sigma = 20 # 标准差
gaussian_noise = np.random.normal(0, sigma, img2.shape)
img_noisy = cv2.addWeighted(img2, 0.5, gaussian_noise.astype(np.uint8),
0.5,0)
```

函数功能：

np.random.normal(0, sigma, img2.shape)

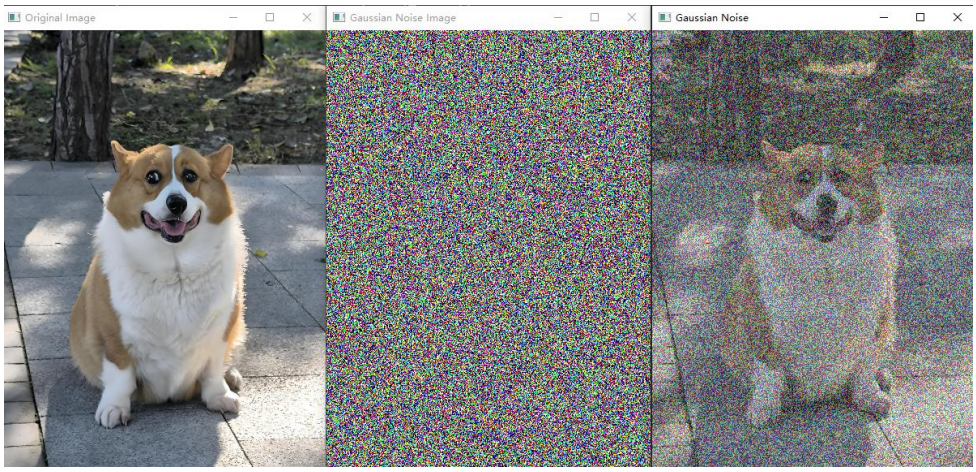
函数生成一个或多个服从指定正态分布的随机数。这在图像处理中可以用来模拟高斯噪声，因为高斯噪声通常被建模为具有零均值和指定标准差的正态分布。

numpy.random.normal(loc, scale, size=None)

loc：分布的均值（mean）。默认为 0。

scale：分布的标准差（standard deviation）。默认为 1。

size：输出数组的形状。如果未指定，输出将是一个单一的随机样本



2.生成椒盐图片

```
# 遍历 img2 的形状
for i in img2.shape:#60,50,3
    # 生成 num_salt 个在 0 到 i-1 之间的随机整数
    random_ints = np.random.randint(0, i - 1, int(num_salt))#450
    # i:从 60 中产生 9 个随机数
    # i:从 50 中产生 9 个随机数
    # i:从 3 中产生 9 个随机数 0 或者 1
    print("---i={}--".format(i))
    print(random_ints)
    # 将生成的随机整数添加到 coords 列表中
    coords.append(random_ints)
    print(coords)
# coords = [np.random.randint(0, i - 1, int(num_salt)) for i in img2.shape]
# print(coords)
img2[coords[0], coords[1], :] = 255   # 白噪声
```

3.在图片上添加文字

在 Python 中使用 Pillow 图像处理库，创建一个可以在图像上绘图的对象，具体函数：

draw = ImageDraw.Draw(pillimg)

ImageDraw 是 Pillow 库中的一个模块，它提供了一组用于在图像上绘制形状（如线条、圆形、矩形等）和文本的函数。

ImageDraw.Draw() 是 ImageDraw 模块中的一个函数，它接受一个图像对象作为参数，并返回一个可以在该图像上绘图的对象。

注意：使用 ImageDraw.Draw 函数，需要将 BGR 格式的颜色转换为 RGB 格式

```
def dramText(img):
    pillimg = Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    draw = ImageDraw.Draw(pillimg)#图像上打印
    #参数 1：字体文件路径，参数 2：字体大小
    font = ImageFont.truetype("simhei.ttf", 40, encoding="utf-8")
    #参数 1：打印坐标，参数 2：文本，参数 3：字体颜色，参数 4：字体
    draw.text((0, 0), "你好", (255, 0, 0), font=font)
    #PIL 图像转 cv2 图像
    cv2charimg = cv2.cvtColor(np.array(pillimg), cv2.COLOR_RGB2BGR)
    return cv2charimg
```